Class 18: Modeling, Experiment Tracking, Evaluation, and Decision-Making

Data-driven Systems Engineering

Roadmap of the class

1. **Modeling foundations**
   Hypothesis class and loss, model families, the scikit-learn interface, `Pipeline`.
2. **Experiment tracking and search**
   What to record, fair comparison, hyperparameter search, Weights & Biases.
3. **Artifacts, versioning, and registry**
   Git versus artifacts, the W&B Registry, promotion and rollback.
4. **Evaluation**
   Metrics that match the task, class imbalance, error inspection.
5. **From prediction to decision**
   Interfaces, the decision rule, decision support, trust.
6. **Practice and wrap-up**
   Repository connections and take-away.

Modeling foundations

What are we choosing when we choose a model, and how do we train it reproducibly?

Hypothesis class and loss, model families, the scikit-learn interface, Pipeline

1. Modeling | 2. Tracking | 3. Registry | 4. Evaluation | 5. Decisions | 6. Practice

What is modeling?

- A model is a function $f : \mathcal{X} \to \mathcal{Y}$ chosen from a hypothesis class $\mathcal{H}$ (e.g. linear functions, decision trees) so that it generalizes to unseen samples.
- Learning selects f by minimizing the empirical risk on the training data,

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i)),$$

  where the loss L quantifies the cost of an error; the quantity of interest is the expected risk on new data.
- The choice of $\mathcal{H}$ encodes assumptions about structure, noise, and decision boundaries, and determines the trade-off among accuracy, interpretability, computational cost, and maintainability.

Questions that precede the choice of a model



Which **decision or prediction** does the model support?



What is the **target space**: a class, a score, or a continuous value (cf. Class 13)?



Which **errors are costly**, and how costly relative to each other?



What are the **latency constraints** for training and inference?



How frequently is the **data distribution** expected to change?

Question 1 of 5: decision or prediction



Which decision or prediction does the model support?

A. An automatic action

- Example: label each bottle as premium or standard on the price list, with no human review.
- The model must output a **reliable class (and probability)** the system can act on.
- Favors models with reliable scores (logistic regression, gradient boosting) and a documented decision rule.

B. Support for a human

- Example: rank wines for a sommelier to taste first.
- Only the **ordering** of cases matters, and the sommelier needs a reason for each ranking.
- Favors ranking metrics (precision at k) and interpretable outputs.

Question 2 of 5: target space



What is the target space: a class, a score, or a continuous value?

A. A class

- Example: premium or standard wine (quality ≥ 7 or not).
- Use a **classifier**; optionally expose `predict_proba` for a score.
- Evaluate with precision, recall, and the confusion matrix, not with error in units.

B. A continuous value

- Example: the wine quality score itself; pasteurization temperature.
- Use a **regressor**; the loss is measured in the units of the target.
- Evaluate with MAE, RMSE, or R^2; the output is the answer itself.

Question 3 of 5: costly errors



Which errors are costly, and how costly relative to each other?

A. A missed positive is worse

- Example: a premium wine sold as standard loses its margin, which costs more than the opposite error.
- Favor **recall**: use class weights or oversample the rare class.
- Accept more false positives, and budget the returns or re-tastings they create.

B. A false alarm is worse

- Example: a standard wine sold as premium triggers returns and damages the brand.
- Favor **precision**: label premium only on strong evidence.
- Accept that some positives will be missed; make this explicit to the stakeholders.

Question 4 of 5: latency



What are the latency constraints for training and inference?

A. Strict, real-time inference

- Example: a response within milliseconds inside an API.
- Favor **light models**: linear models, shallow trees, a small ensemble.
- Preprocessing must be cheap, and the full `Pipeline` must be loaded once at start-up.

B. Offline, batch scoring

- Example: scoring every bottle of a new vintage in one batch.
- Large ensembles and **extensive hyperparameter search** are affordable.
- The limit is the total training budget, which must be tracked per run.

Question 5 of 5: distribution change



How frequently is the data distribution expected to change?

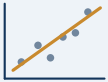
A. Stable distribution

- Example: a controlled laboratory measurement.
- A **fixed model** retrained on a schedule is sufficient.
- Monitor a few input statistics and the error, and register each retrained version.

B. Fast-changing distribution

- Example: a sensor stream with seasonal or process drift.
- Favor **online or incremental models** and drift detection (Class 19).
- Evaluate on the most recent data, not on a random split of the whole history.

Model families in context: supervised learning



Linear models

Efficient, interpretable, strong baselines.

Logistic regression
Bishop (2006), §4.3



Kernel models

Maximum margin; non-linear through kernels.

Support vector machine
Cortes & Vapnik (1995)



Instance-based

Predict from the closest training samples.

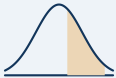
k-nearest neighbours
Cover & Hart (1967)



Trees and ensembles

Tabular data, interactions, boosting.

Random forest
Breiman (2001)



Probabilistic models

Explicit uncertainty and structure.

Naive Bayes
Domingos & Pazzani (1997)



Neural networks

Large unstructured data, highly non-linear.

Multilayer perceptron
Rumelhart et al. (1986)

Model families in context: beyond the supervised toolbox



Clustering

Groups of similar samples, no labels.

k-means
Lloyd (1982)



Dimension reduction

Fewer dimensions that preserve variance.

PCA
Jolliffe (2002)



Anomaly detection

Flags samples unlike the normal data.

Isolation forest
Liu et al. (2008)



Sequence models

Temporal dependence in the data.

Hidden Markov model
Rabiner (1989)



Online learning

Any family, updated incrementally (Class 19).

Hoeffding tree
Domingos & Hulten (2000)

Start simple.
Build a baseline first, then justify each step in complexity.

scikit-learn as an architecture: one interface for every model

Estimator	`fit(X, y)`	Learns from data and stores the result in attributes ending with an underscore (e.g. `coef_`).
Predictor	`predict(X)`	Maps new samples to labels or values; classifiers also expose `predict_proba`, and all predictors expose `score`.
Transformer	`transform(X)`	Rewrites the features (scaling, encoding, imputation); `fit_transform` is the shortcut for the training split.
Meta-estimator	`fit / predict`	Wraps other estimators and exposes the same interface (`Pipeline`, `GridSearchCV`).

The fit / predict contract

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)       # learns mean, s.d.
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)          # same parameters

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train_s, y_train)                # learning happens here

y_pred = model.predict(X_test_s)             # labels (tau = 0.5)
y_prob = model.predict_proba(X_test_s)[:, 1]  # scores
```

Pipeline: preprocessing and model as a single estimator

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
models = {
    "logistic": LogisticRegression(C=1.0, max_iter=1000),
    "svm": SVC(kernel="rbf", C=10, gamma="scale"),
    "forest": RandomForestClassifier(n_estimators=200, max_depth=8),
}
for name, clf in models.items():
    pipeline = Pipeline([("scaler", StandardScaler()),
                         ("model", clf)])
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
```

Experiment tracking and search

How do we compare models fairly and keep a record of every run?

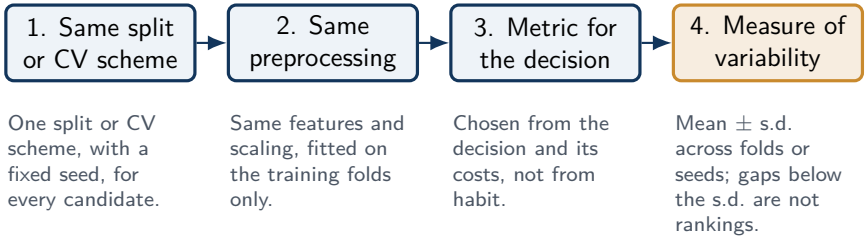
What to track, sources of bias, hyperparameter search, Weights & Biases

1. Modeling | 2. Tracking | 3. Registry | 4. Evaluation | 5. Decisions | 6. Practice

What must be tracked, and why

Dataset version or extraction date	Allows a result to be reproduced on the same data.
Feature pipeline version	Separates the effect of preprocessing changes from that of model changes.
Hyperparameters and random seeds	Identify the exact configuration that produced a result.
Metrics, confusion matrices, artifacts	Allow the trained model, not only a number, to be audited.
Code commit or notebook snapshot	Allows the logic behind a run to be replayed later (Class 17).

A fair comparison: hold everything else fixed



Why it matters

A difference between two models is evidence only if the data, the preprocessing (unless preprocessing is the experiment), and the metric are identical, and if the difference exceeds the run-to-run variability.

Sources of bias and their solutions

Source of bias	Solution
Test-set reuse. Reusing the test set until it becomes part of tuning (model-selection leakage).	Tune on a validation split or with cross-validation; use the test set once, at the end.
Hidden state. Hidden notebook state and untracked edits.	Log commit, configuration, seed, and data version with every run; restart the kernel and run all cells.
Selective reporting. Reporting only the best run.	Report all runs and the search budget; give mean $\pm$ s.d. over folds or seeds.

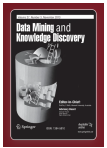
Hyperparameter search strategies

Manual tuning	Small problems; useful to build intuition.
Grid search	Exhaustive evaluation of a small, structured search space.
Random search	Better coverage when only a few of many hyperparameters matter (Bergstra and Bengio, 2012).

Optimization-based search: the validation score is treated as a black-box function.

Bayesian optimization	A surrogate model of the score (Gaussian process, TPE) guides the next trial; suited to expensive models.
Genetic algorithms	A population of configurations evolves through selection, crossover, and mutation.
Other heuristics	Simulated annealing, particle swarm, and bandit-based early stopping (successive halving, Hyperband).

Case study: how much does tuning help a decision tree?



Mantovani, Horváth, Rossi, Cerri, Barbon Junior, Vanschoren, de Carvalho (2024). *Better trees: an empirical study on hyperparameter tuning of classification decision tree induction algorithms*. Data Mining and Knowledge Discovery, 38(3), 1364–1416.

Experimental design

- **Models and data:** CART and C4.5; 94 OpenML classification datasets.
- **Six techniques:** random search, genetic algorithm, particle swarm, estimation of distribution, model-based optimization, Irace.
- **Protocol:** nested CV (10 outer, 3 inner folds); same partitions and seeds; 900 evaluations.
- **Baseline:** default hyperparameters.

Findings

- Significant gains over the defaults: **CART** in about two thirds of the datasets, **C4.5** in one third.
- **Irace** was the best technique; few evaluations were enough.
- **A small subset of hyperparameters** explains most of the gain (CART: `minsplit`, `minbucket`).
- **Defaults can already be good**.

What is Weights & Biases (W&B)?

A platform for **tracking, comparing, and versioning machine learning work**. A few lines of the `wandb` Python library send metrics, configuration, and files to a web dashboard, which can be hosted by W&B or self-hosted. Website: wandb.ai.

Run	One execution of a script or notebook, with a unique identifier.
Config	The inputs of a run: hyperparameters, seed, dataset version.
Metrics	Values logged with `run.log`: scores, losses, timings, plots.
Sweep	An automated hyperparameter search (grid, random, or Bayesian).
Artifact	A versioned file or directory (dataset, model) with its lineage.
Registry	Curated collections of approved model versions.

In one sentence

Git remembers how the code changed; W&B remembers what each run of that code produced.

Alternatives: MLflow (open source, self-hosted), TensorBoard (local dashboards), Comet, ClearML.

W&B case study: search space and protocol

Search space of a gradient boosting regressor:

Hyperparameter	Values
`n_estimators`	100, 200, 300
`max_depth`	3, 5, 7
`learning_rate`	0.05, 0.1, 0.2
`min_samples_split`	2, 5, 10

Protocol:

- Data: red wine quality, 1,599 samples; target `quality`.
- Split: 75% training (1,199 wines), 25% test (400 wines), fixed seed.
- Grid search: $3^4 = 81$ configurations (one W&B run each) $\times$ 5-fold cross-validation $=$ 405 fits, scored by R^2 on the training split; each run then refits once to save its artifact.
- The test split is used once, after selection.

Implemented in the `wine-wandb` project (`configs/sweep.yaml`, `src/wine_wandb/sweep.py`); no numerical result of the sweep is shown until it has been executed.

W&B: the record behind an experiment

The experiment tracker links an explicit hypothesis to a reproducible configuration and measurable evidence. An example record for the wine-quality exercise:

Field	Example logged information
`run.config`	model family, grid, random seed, cross-validation scheme
`run.summary`	mean CV R^2, CV standard deviation, fitting time
`git commit`	commit SHA for training and preprocessing code
`dataset`	dataset identifier, checksum or immutable artifact version
`model artifact`	serialized `Pipeline` (preprocessing + predictor)

- Log candidate performance using cross-validation on the training partition; evaluate the chosen model only once on the untouched test partition.
- **Do not confuse** experiment logs, artifact versions, and model approval: they answer different questions.

W&B: minimal experiment tracking in Python

```python
import wandb
from sklearn.model_selection import cross_val_score

config = {"model": "GradientBoostingRegressor", "seed": 42}
with wandb.init(project="wine-quality", config=config) as run:
    scores = cross_val_score(pipeline, X_train, y_train, cv=5)
    run.log({"cv/r2_mean": scores.mean(),
             "cv/r2_std": scores.std()})
    artifact = wandb.Artifact("wine-pipeline", type="model")
    artifact.add_file("pipeline.joblib")
    run.log_artifact(artifact)
```

Artifacts, versioning, and registry

How does a trained model become an approved, versioned, reversible release?

Git versus artifacts, the W&B Registry, promotion and rollback

1. Modeling | 2. Tracking | **3. Registry** | 4. Evaluation | 5. Decisions | 6. Practice

Git, tracking, artifacts, and registry: distinct roles

Component	Stores or identifies	Primary question
GitHub	source, tests, configuration, CI workflows	Which code produced this run?
W&B Runs	parameters, metrics, environment and run identifiers	What happened during training?
W&B Artifacts	immutable versions of data and model files	Which exact file was produced or used?
W&B Registry	curated collections and approved model versions	Which version is eligible for operational use?

Key distinction

Versioning a `model.pkl` filename is not enough: a deployed model needs *code* + *data* + *preprocessing* + *environment* + *trained artifact* + *approval record*.

Why not commit trained binaries directly to Git?

Git is excellent for

- code and configuration diffs;
- review and pull requests;
- reproducible workflow definitions.

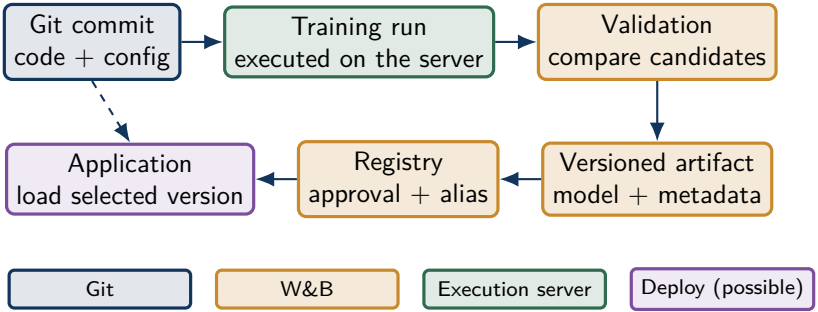
Model artifacts additionally need

- versioned object storage and integrity hashes;
- links to runs and input datasets;
- approval, access control, and rollback history.

Practical exception

Tiny demonstration binaries may be committed for a self-contained example, but this is not a scalable production artifact strategy. Git LFS manages large files differently; it does not by itself provide experiment tracking or model governance.

From an experiment to an approved model



W&B: publish an artifact to the Registry

```python
import wandb
import joblib

joblib.dump(pipeline, "pipeline.joblib")
with wandb.init(project="wine-quality") as run:
    artifact = wandb.Artifact("wine-pipeline", type="model")
    artifact.add_file("pipeline.joblib")
    logged = run.log_artifact(artifact)
    logged.wait()
    run.link_artifact(
        logged,
        target_path="wandb-registry-model/wine-quality",
    )
```

Promotion, rollback, and a model release contract

1. **Candidate:** compare runs on validation data; preserve the full candidate record.
2. **Gate:** test metrics, stability, inference latency, feature schema, and reproducibility against agreed criteria.
3. **Approval:** link the accepted artifact to a Registry collection; assign a documented alias such as `champion` when appropriate.
4. **Release:** application records the resolved immutable artifact version and environment.
5. **Rollback:** repoint the release to the preceding approved artifact and revalidate compatibility.

Discussion

Why is selecting the highest CV score insufficient to promote a model to production?

Evaluation

Which metric tells the truth about this task?

Metrics, class imbalance, inspecting the errors

1. Modeling | 2. Tracking | 3. Registry | **4. Evaluation** | 5. Decisions | 6. Practice

Evaluation metrics must match the task

Tracking identifies which run performed best; it does not establish whether that run is adequate for the decision it supports.

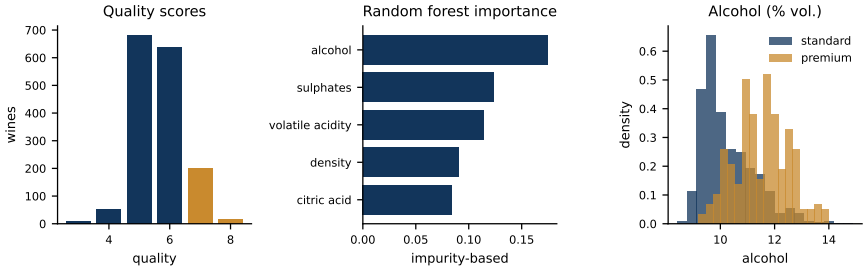
Classification (premium wine)

- **Precision** $= \frac{TP}{TP+FP}$ and **recall** $= \frac{TP}{TP+FN}$, from the counts of true and false positives and negatives.
- **PR-AUC:** area under the precision–recall curve; summarizes all possible cut-offs.
- **Accuracy** alone is misleading when classes are imbalanced.
- **Calibration:** needed when an action depends on the predicted probability.

Regression (wine quality)

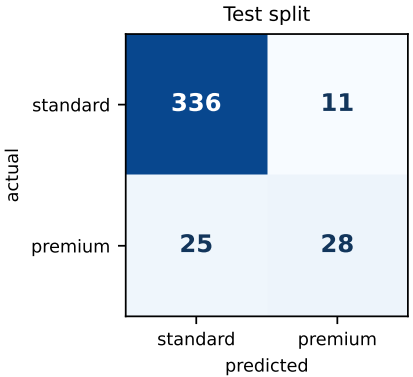
- **MAE** $= \frac{1}{n}\sum_i |y_i - \hat{y}_i|$ and **RMSE** $= \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$, in the unit of the target (quality points).
- **RMSE** penalizes large errors more than MAE does.
- R^2**:** share of the variance explained; 0 for a model that always predicts the mean.
- **Residuals** show where the model fails, for example at extreme scores.

The wine dataset at a glance



- **Imbalance:** 217 of 1,599 wines (13.6%) have quality ≥ 7 and are labelled *premium*; the same split as the regression exercise gives 1,199 training and 400 test wines (164 and 53 premium).
- **Importance:** alcohol, sulphates, and volatile acidity account for 41% of the impurity-based importance of the random forest; importance measures association, not causal effect.
- **Separation:** premium wines have a higher median alcohol content (11.6 versus 10.0% vol.), but the distributions overlap, so no single feature decides the label.

Inspecting the errors



Test split (400 wines), random forest, predicted classes.

$$\text{precision} = \frac{28}{28+11} = 0.72, \qquad \text{recall} = \frac{28}{28+25} = 0.53$$

$$\text{accuracy} = \frac{336+28}{400} = 0.91$$

- 25 of the 53 premium wines are missed (FN), while 11 standard wines are labelled premium (FP).
- The accuracy of 91% sounds good, but always answering "standard" already scores 86.75%; the confusion matrix exposes the missed premium wines.
- Whether 25 missed wines are acceptable depends on the cost of a missed premium wine relative to a wrong label: a business input, not a property of the model.

Accuracy fails under imbalance: alternatives

Wine example

Always answering "standard" scores 86.75% accuracy with a recall of 0. The random forest reaches 91%, only 4 points more, yet misses almost half of the premium wines.

Metric	What it measures	Always "standard"	Random forest
Accuracy	Share of correct answers; dominated by the majority class.	0.87	0.91
F1-score	Harmonic mean of precision and recall, $2PR/(P+R)$; ignores true negatives.	0	0.61
ROC-AUC	Chance that a random premium wine outscores a random standard one; independent of any cut-off.	0.50	0.94
PR-AUC	Area under the precision–recall curve; chance level is the share of positives.	0.13	0.75

- F1 is computed from the predicted classes and weighs both errors equally.
- ROC-AUC (0.94) exceeds PR-AUC (0.75) because true negatives dominate; with rare positives, prefer PR-AUC.
- None encodes the cost of each error: that is a business input.

From prediction to decision

How does a score become an action that people can trust?

Interfaces, the decision rule, decision support, where trust breaks

1. Modeling | 2. Tracking | 3. Registry | 4. Evaluation | **5. Decisions** | 6. Practice

A registered model creates value through an interface

- A registered trained model that cannot be invoked or interpreted has limited practical value, regardless of its offline metrics.
- An interface makes predictions available to users, services, and operational workflows.
- The interface also determines trust: input validation, response format, and clarity matter as much as the metric used to select the model.

A prediction is not a decision

- A premium score suggests that a wine deserves a tasting, not a final grade; a process alert suggests investigation, not automatic shutdown.
- A classifier estimates $P(y = 1 \mid \mathbf{x})$; the class it outputs applies a decision rule, $\hat{y} = \mathbf{1}[\hat{p}(\mathbf{x}) \geq 0.5]$.
- `predict` makes that rule implicit: it assumes both errors cost the same.

In this class

We adopt the predicted class as the decision. With unequal error costs, the cut-off would be a separate design choice.

Example: a decision-support interface for wine



- The interface loads the registered `wine-premium` artifact, so preprocessing cannot diverge from training, and shows the predicted label together with the estimated probability.
- The predicted class is the decision; the probability tells the user how confident the model is.
- The interface should identify the exact artifact version it loaded.

A decision-support interface: requirements and failure modes

What it must provide

- What the model does and does not do.
- Inputs that are meaningful domain variables, not technical internals.
- Outputs that support action: label, probability, explanation.
- Provenance: data version, model artifact version, monitoring signals.

Where trust breaks down

- Uncertainty hidden behind one precise-looking number.
- Inputs that violate the assumptions of training.
- A decision rule adopted by default and never revisited.
- A demonstration model that looks authoritative without validation.

Practice and wrap-up

How do the pieces fit together in a real repository?

Repository connections and take-away

1. Modeling | 2. Tracking | 3. Registry | 4. Evaluation | 5. Decisions | 6. Practice

Repository connections

Repo activity

- The `wine-wandb` project (a separate repository): the complete wine-quality project used throughout this class, with two tasks on the same dataset.
- `configs/`: `baseline.yaml` and `sweep.yaml` (regression), `classifier.yaml` (premium wine).
- `train.py`, `sweep.py` and `train_classifier.py`: tracked runs; each logs the dataset checksum, the Git commit, and a pipeline artifact (`wine-pipeline`, `wine-premium`).
- `evaluate.py`, `register.py`, `figures.py`: one-time test evaluation, registry promotion, and the figures of this class.

Papers and materials

- Bergstra, J., Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.
- Bergstra, J., Yamins, D., Cox, D. (2013). Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. *Proceedings of ICML*, PMLR 28(1), 115–123.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Saito, T., Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432.
- Mantovani, R. G., Horváth, T., Rossi, A. L. D., Cerri, R., Barbon Junior, S., Vanschoren, J., de Carvalho, A. C. P. L. F. (2024). Better trees: an empirical study on hyperparameter tuning of classification decision tree induction algorithms. *Data Mining and Knowledge Discovery*, 38(3), 1364–1416.
- Niculescu-Mizil, A., Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of ICML*, 625–632.
- Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78–87.
- Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly.
- scikit-learn User Guide: tuning hyperparameters; model evaluation.
- Tool documentation: Weights & Biases; MLflow.

Papers and materials: model families

Papers and materials

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Logistic regression: §4.3.)
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Cortes, C., Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- Cover, T., Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- Domingos, P., Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning*, 29, 103–130.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.
- Liu, F. T., Ting, K. M., Zhou, Z.-H. (2008). Isolation forest. *Proceedings of ICDM*, 413–422.
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257–286.
- Domingos, P., Hulten, G. (2000). Mining high-speed data streams. *Proceedings of KDD*, 71–80.
- Rumelhart, D. E., Hinton, G. E., Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

Take-away message

A model is selected by reproducible experiments,
preserved as a versioned artifact, and approved in a Registry.

Its operational decision also depends on
evaluation evidence, the release configuration, and error costs.

Thank you for your attention.

Next: Class 19 — online machine learning and concept drift.